# Optimal Freelancing

AlgoExpert · Easy · Greedy Algorithms

# The problem

- Each job takes one day and has a deadline (days left) and a payment.
- Working at most 7 days, one job per day, maximise profit.
- Greedy: take jobs by payment descending and place each in the latest free day on or before its deadline.

# Intuition

- Greedy by payment: take the most valuable jobs first and schedule each as late as its deadline allows.

# Brute force vs optimal

- Brute force: try every subset and order of jobs → exponential
- Greedy: highest pay first, as late as possible → O(n log n)
- Scheduling late keeps early days free for tight deadlines

# Approach

- 1. Sort jobs by payment, highest first
- 2. Keep seven day slots
- 3. Place each job in the latest free day before its deadline
- 4. Sum the payments of placed jobs

# Worked example

| job | C | D | A | B |
|---|---|---|---|---|
| deadline | 3 | 2 | 1 | 2 |
| pay | 2 | 2 | 1 | 1 |

days 1–7 (index = day − 1)

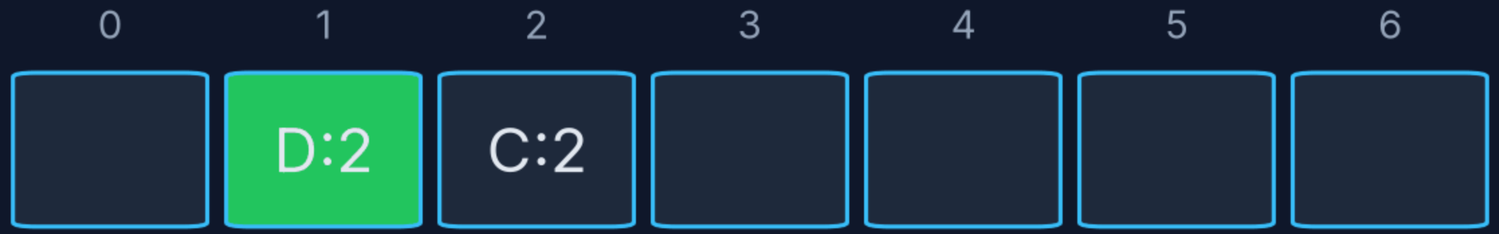



Job C (pay 2, deadline 3) → day 3; earned 2

# Worked example

| job | C | D | A | B |
|---|---|---|---|---|
| deadline | 3 | 2 | 1 | 2 |
| pay | 2 | 2 | 1 | 1 |

days 1–7 (index = day − 1)




Job D (pay 2, deadline 2) → day 2; earned 4

# Worked example

| job | C | D | A | B |
|---|---|---|---|---|
| deadline | 3 | 2 | 1 | 2 |
| pay | 2 | 2 | 1 | 1 |

days 1–7 (index = day − 1)




Job A (pay 1, deadline 1) → day 1; earned 5

# Worked example

| job | C | D | A | B |
|---|---|---|---|---|
| deadline | 3 | 2 | 1 | 2 |
| pay | 2 | 2 | 1 | 1 |

days 1–7 (index = day − 1)




Job B (pay 1, deadline 2) → no free day, skip; earned 5

# Java solution

```java
public class OptimalFreelancing {
    public static int optimalFreelancing(List<Map<String, Integer>> jobs) {
        final int days = 7;
        boolean[] taken = new boolean[days];
        int profit = 0;
        List<Map<String, Integer>> sorted = new java.util.ArrayList<>(jobs);
        sorted.sort((a, b) -> Integer.compare(b.get("payment"), a.get("payment")));
        for (Map<String, Integer> job : sorted) {
            for (int d = Math.min(job.get("deadline"), days) - 1; d >= 0; d--) {
                if (!taken[d]) { taken[d] = true; profit += job.get("payment"); break; }
            }
        }
        return profit;
    }
}
```

# Complexity

- Time: O(n log n)
- Space: O(1) (7 slots)

# Edge cases & pitfalls

- Deadlines beyond 7 → cap at day 7
- Equal payments: any order works
- No jobs → 0

# Interview tips & follow-ups

- Explain why 'as late as possible' matters
- Follow-up: n days instead of 7 → union-find for free slots

# Recap

- Greedy by payment: take the most valuable jobs first and schedule each as late as its deadline allows.
- Complexity: O(n log n)